Vehicle Rental System

The **Vehicle Rental System** provide a seamless experience for customers to rent vehicles while allowing administrators to manage vehicles, users, bookings. Implement the application using Repository Pattern and Entity framework core and Asp.net core MVC. **Use code first Approach to design a Schema**

1. User Management Scenarios

User should be able to Register and Login to the application. Design user class with the following properties

- **UserId**
- **Name**
- **Email**
- **Password**
- **Role ("Admin" or "Customer")**
- **// Navigation Property**
- **List<Booking> Bookings**

2. Vehicle Management Scenarios

Admin should be able to add vehicles, update vehicles and delete vehicles. Design the vehicle class with following properties and methods

Class: Vehicle

- VehicleId
- Model
- Brand
- Type // SUV, Sedan, Bike, etc.
- RentalPricePerDay
- IsAvailable

}

Interface Methods (IVehicleRepository)

- Task<IEnumerable<Vehicle>> GetAllVehiclesAsync();
- Task<Vehicle> GetVehicleByIdAsync(int vehicleId);
- Task AddVehicleAsync(Vehicle vehicle);
- Task UpdateVehicleAsync(Vehicle vehicle);
- Task DeleteVehicleAsync(int vehicleId);
- Task<IEnumerable<Vehicle>> GetAvailableVehiclesAsync();

3. Booking Management Scenarios

Customer should be able to book and update and delete Bookings

Class: Booking

- BookingId
- VehicleId
- UserId
- StartDate
- EndDate
- TotalAmount
- Vehicle Vehicle // Navigation Property
- User User // Navigation Property

}

Interface Methods (IBookingRepository)

- Task<IEnumerable<Booking>> GetAllBookingsAsync();
- Task<Booking> GetBookingByIdAsync(int bookingId);
- Task AddBookingAsync(Booking booking);
- Task<IEnumerable<Booking>> GetBookingsByUserIdAsync(int userId);
- Task<IEnumerable<Booking>> GetBookingsByVehicleIdAsync(int vehicleId);
- Task DeleteBookingAsync(int bookingId);

4. Custom Exception Scenarios

Implement following custom exceptions

Custom Exceptions

- VehicleNotFoundException
- Booking Conflict Exception

5. Relationship Between Classes

User (1) <----> (M) Booking
Vehicle (1) <----> (M) Booking
Booking (1) <----> (1) Payment
One User can have multiple Bookings
One Vehicle can have multiple Bookings
One Booking has only one Payment